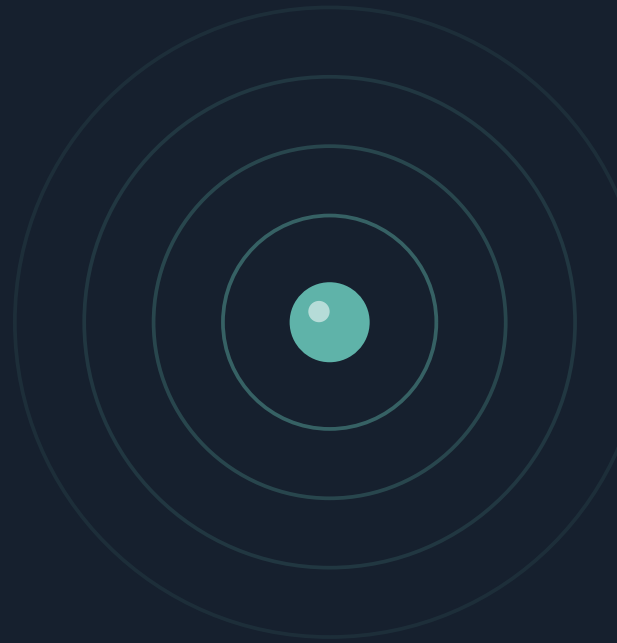




## SESSION SUMMARY

---

# Vibe Coding and Building an AI MVP

How a non-coder planned, built, voice-enabled, connected and published a personal CRM dashboard in one sitting.

---

### SESSION DATE

September 19, 2026

### PROGRAM

Day 6, second session

### TOOL USED

Lovable

### MENTOR

Harshith Vaddiparthi

### HOST

Sumedha

### FORMAT

Live build, then Q and A

---

# Contents

|                                                    |    |
|----------------------------------------------------|----|
| Session at a Glance . . . . .                      | 2  |
| Opening and Housekeeping . . . . .                 | 3  |
| The Mindset Before the Build . . . . .             | 4  |
| Core Concepts . . . . .                            | 4  |
| Choosing a Tool . . . . .                          | 5  |
| The Building Blocks of an App . . . . .            | 6  |
| Ready-Made Design: shadcn/ui and tweakcn . . . . . | 7  |
| The Product Idea . . . . .                         | 9  |
| Build Stage: Foundation in Plan Mode . . . . .     | 9  |
| Build Stage: Features in Phases . . . . .          | 11 |
| Build Stage: Applying the Design System . . . . .  | 12 |
| Build Stage: The Jarvis Voice Agent . . . . .      | 13 |
| Build Stage: Integrations . . . . .                | 15 |
| API Keys, Credits and Cost . . . . .               | 16 |
| Publishing . . . . .                               | 17 |
| Turning This Into Income . . . . .                 | 18 |
| Questions From the Chat . . . . .                  | 19 |
| Common Mistakes and Misconceptions . . . . .       | 20 |
| Action Steps . . . . .                             | 20 |
| Key Takeaways . . . . .                            | 21 |
| Closing and What Comes Next . . . . .              | 21 |
| Glossary . . . . .                                 | 22 |

## Session at a Glance

This was the second session of Day 6. The morning session covered voice agents. This session covered vibe coding. The mentor, Harshith Vaddiparthi, built a working product live on screen. He started from a blank prompt and ended with a published app.

The product was a personal productivity CRM dashboard. It has Google sign-in, a database, a clean dashboard layout, a voice assistant called Jarvis, and live connections to Google Calendar and Gmail. The whole build took about 3 hours. The mentor does not code. Every step was done by prompting.

The session had one goal. Teach a method that works for any product idea and on any vibe coding tool. The dashboard was only the example.

### What the Session Covered

- What vibe coding is and what an MVP is.
- The loop that every build follows: prompt, generate, review, refine.
- The parts of an app: front end, back end, database, APIs, hosting.
- The stack the mentor recommends: Next.js, Node.js, Supabase.
- Ready-made design: shadcn/ui for components and tweakcn for themes.
- How to plan before building, and how to split features into phases.
- How to add a voice agent that takes actions inside the app.
- How to connect Google Calendar, Gmail, WhatsApp and other tools.
- How API keys and credits work, and how to publish.
- How to sell this kind of work to clients.

### What Was Built by the End

- A landing page with a Get Started button and Google sign-in.
- A dashboard with a collapsible sidebar and an analytics layout.
- Pages for tasks, projects, calendar, notes, goals, reminders, contacts, mail and settings.
- A right-side panel with the Jarvis orb. It listens, acts and answers by voice.
- A live test where Jarvis added a task with a due date from a spoken request.
- Google Calendar events and the Gmail inbox showing inside the dashboard.
- A published, live URL.

#### THE MAIN MESSAGE

The building blocks of the internet are open source. If you know which blocks to use and how to ask for them, you can build almost anything by prompting. The skill is in the planning, the reviewing and the choice of tools.

---

# Opening and Housekeeping

## Where This Session Sits in the Program

Sumedha hosted. She reminded the group of what had been covered so far: prompting, MCPs, automations, and voice agents that same morning. Vibe coding was next. She asked who had product ideas they could not build because they were not technical. Many people did. One learner had 20 to 25 ideas written in a diary. She said this session was the key to building them.

## Format for the Day

- A learning session of 2.5 to 3 hours. One product built from scratch to MVP.
- Then breakout rooms for questions and answers.
- No workbook for this session.
  - Learners will get more workbooks and more practice later.
  - The team did not want learners to use up their credits now.
  - Credits should be saved for the hackathon.
- Questions during the session went into the Q and A panel. Co-mentors answered them live.
- Questions in the breakout rooms went into Slido. Mentors took them in order of popularity.
- Questions from the morning voice agent session were also welcome.

## Co-Mentors for the Day

- Sandesh. With the group since day 1. Also seen in the LMS.
- Ritesh. With the group since day 1. Has run breakout rooms before.
- Amlan. A founder. His first session with this group.
- Simran. Has taken earlier sessions and breakout rooms.
- Shaswat. Met the group during the automation section.

## About the Mentor

Most learners already knew Harshith from the daily office hours, held from 6:00 PM to 7:00 PM. Sumedha told everyone else to watch those recordings.

- AI consultant. Co-founder and CTO at an AI startup.
- AI mentor at Outskill.
- Runs Harshith Ventures, a small fund that writes angel checks for early startups.
- Previously exited an AI startup that did financial analysis.
- Featured in Forbes and Bloomberg.
- Website: [harshith.com](https://harshith.com). His blog there is written and posted by AI after he approves the tone.
- Runs several newsletters on autopilot.
- Builds dashboards and personal agents for clients. This is his most common client project.

He stressed one point about himself many times. He is not a coder. 2 to 3 years ago he had to ask developer friends to build even a simple website. He did not know what GitHub, Vercel or hosting

were. He said people assume he is technical only because he uses words like MVP and API.

---

## The Mindset Before the Build

### Opening Demo: His Own Jarvis

Before teaching anything, Harshith showed what is possible. He opened his own desktop assistant, which he calls Jarvis, and spoke to it. He asked it to greet the learners. It answered in a cloned female voice and welcomed everyone to the build. He is a fan of Iron Man and wanted a Tony Stark style assistant. He said he could never have built it before AI coding tools. It took him about 4 weeks.

### Vibe Coding as Expression

He called vibe coding a form of human expression. If you can think of it, you can make it. He said his consulting, his fund and his newsletters all exist because of these tools.

### Watch First, Build Later

He asked learners many times not to open Lovable and build along with him. The reasons:

- The value is in how he approaches the build and how he words his prompts.
- Anyone building at the same time would miss a package, a resource or a key step.
- He wasted a lot of tokens over 3 years because these principles were not clear to him.
- Clear principles save tokens, save time and give better results.
- The recording, the prompts, a slide deck and his notes would all be shared after the session.

He also asked for an active chat. He would take product ideas, feature ideas and design choices from the group. He described himself as a vibe coder with a bit of taste, and said the group would have better ideas than he did.

---

## Core Concepts

### What Vibe Coding Is

His definition: you talk to the AI about what to build, and it builds it for you. You can speak or type. A learner called it text to code. Another pointed out that it is just prompting. He agreed with both. If you can prompt, you can build.

### What an MVP Is

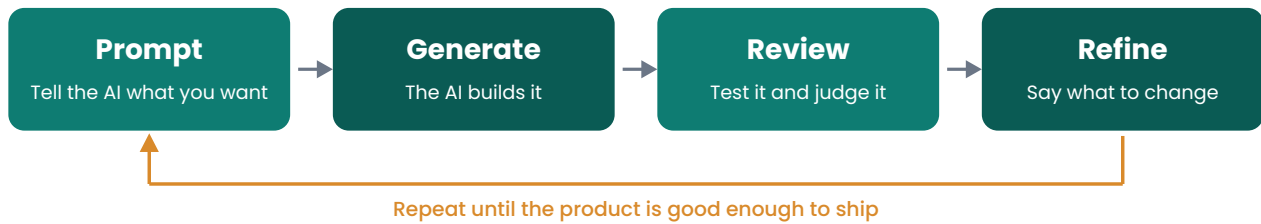
MVP stands for minimum viable product. His definition: the most basic, working prototype that you can ship. Ship means to deliver it to real users. It can also be called a proof of concept. It is the first version you take to market.

### Why Not Build the Final Product Straight Away

A learner asked this. His answer: no large product was built in one go. Zomato, Cred and Instagram all went through many rounds of change. You take small steps. Another learner put it as MVP plus enhancement, plus enhancement, plus enhancement. He liked that wording.

## The Build Loop

Every build follows the same loop. The first output is never the best one. You go around the loop many times.



- **Prompt.** Tell the AI what you want.
- **Generate.** The AI writes the code and builds it.
- **Review.** Open the preview. Click things. Decide if it is what you wanted.
- **Refine.** Tell the AI what is wrong or missing. That becomes the next prompt.

He said the review and refine steps are where taste and judgment matter. This is also where he asked for the group's help during the session.

## Choosing a Tool

### Tools Named in the Session

| Tool                                 | What was said about it                                                     |
|--------------------------------------|----------------------------------------------------------------------------|
| Lovable                              | The tool used for this session. He used it only because he has an account. |
| v0 by Vercel                         | The original. It first made components only.                               |
| Bolt by StackBlitz                   | One of the first to build full websites. He used it a lot.                 |
| Cursor                               | Named as an option.                                                        |
| Replit                               | Named as an option.                                                        |
| Kiro by Amazon                       | Named as an option.                                                        |
| Antigravity by Google                | Named as an option.                                                        |
| Emergent, same.dev, Base44, Anything | Named as options.                                                          |
| Claude Code                          | A different kind of tool. More of a desktop application.                   |

**METHOD OVER TOOL**

The method works on any of these tools. Lovable was used only for convenience. Learn the approach, then apply it on the tool you like.

## The Building Blocks of an App

Before writing any prompt, he moved to a whiteboard. He called this step wireframing. You map out what you will build and the parts it needs. He noted that this work used to be a full job for a business analyst.

### The Layers

- **Front end.** What the user sees and clicks.
- **Back end.** The logic that runs behind the screen.
- **Database.** What the app remembers.
- **APIs.** The connectors between the layers and to outside services.
- **Hosting.** Where the finished app lives online.
- **CMS.** A content management system. He treats it as a separate tool you connect, not a core layer.

He said non-coders find the front end easy because they can see it. The back end is hard because they cannot see what is going on. That is why he explained the back end and the stack first. In the actual build, front end and back end were done in parallel.

### Front End: Next.js

- A framework made by Vercel. Vercel is a company that makes hosting tools and v0.
- For a non-coder, think of it as a coding language that is very good for modern websites.
- It is built on React, which is based on JavaScript.
- It is good for SEO, handles styling well and is easy to host.
- It connects well to back-end systems and to APIs. He compared it to the best LEGO blocks.
- It is used very widely. AI models have seen a lot of it in training.
  - So AI tools write it well.
  - So AI tools fix bugs in it fast.
- Its documentation is updated often. You can paste a documentation page into the chat when the AI is stuck.
- You do not need to know how to write it. You only need to know it exists and when to ask for it.

### Back End: Node.js

The chat listed many back-end options: Python, .NET, Java, Spring Boot and more. He joked that the list scared him. His pick as a non-coder is Node.js. It is easy to work with and easy to prompt. A learner noticed it is the same Node.js they installed a day earlier to run MCPs. He built on that. If Node.js can run an MCP, that MCP can also be connected to the dashboard.

## Database: Supabase

- The database is the memory of the app.
  - A fitness app stores your name, age, height and weight there.
  - Instagram stores your followers and who you follow there.
  - His newsletter tool stores each subscriber, opens, unsubscribes and bounces there.
- A learner suggested Google Sheets. He said no. It is not a proper database for an app.
- Supabase is his pick. Most of the chat had already named it.
- It is open source. It has over 110,000 stars on GitHub. You can run it free on your own server.
- It includes authentication and file storage.
- It is popular with vibe coders and works well with integrations.
- Firebase is also an option.
- The name is wordplay on database. The company is backed by Y Combinator.

## The Recommended Stack in One Table

| Layer      | Pick      | Plain meaning                                          |
|------------|-----------|--------------------------------------------------------|
| Front end  | Next.js   | Builds the screens. Best fit for the modern web.       |
| Back end   | Node.js   | Runs the logic. Connects to almost everything.         |
| Database   | Supabase  | Stores users and data. Also handles sign-in and files. |
| Components | shadcn/ui | Ready-made interface parts.                            |
| Theme      | tweakcn   | A full design system on top of shadcn/ui.              |
| Connectors | APIs      | Link the layers and outside services.                  |

One learner said she understood it more or less. He did not accept that and explained it again. He added that many of these terms sound harder than they are. If the database idea is still unclear, watching the build makes it clear.

## Ready-Made Design: shadcn/ui and tweakcn

### shadcn/ui: The Component Library

shadcn/ui is an open-source library of interface parts. It is named after its creator, who goes by shadcn online and works at Vercel. He saw that many people were building dashboards and made a full set of parts for them. UI means user interface, the part of an app you see.

- A component is one piece of an interface. A button, a card, an accordion, a sidebar.
- The site shows a preview and the code for each part. A non-coder only needs the preview.
- You do not describe a login page in long detail. You ask for the shadcn card or login component by name.



- This saves time and saves credits, because the AI is not designing from zero.
- Everything is modular. Any part can be used in any project.

## Blocks and the Layout Choice

The Blocks section holds full layouts that are already built, such as sidebars and whole dashboards. He first picked the block called Sidebar 7, which he had used before. Later he changed to the first dashboard block, because it includes an analytics area and shows the result better. He said either one would work. The names are only labels on the site. They are not coding terms.

## tweakcn: The Theme Layer

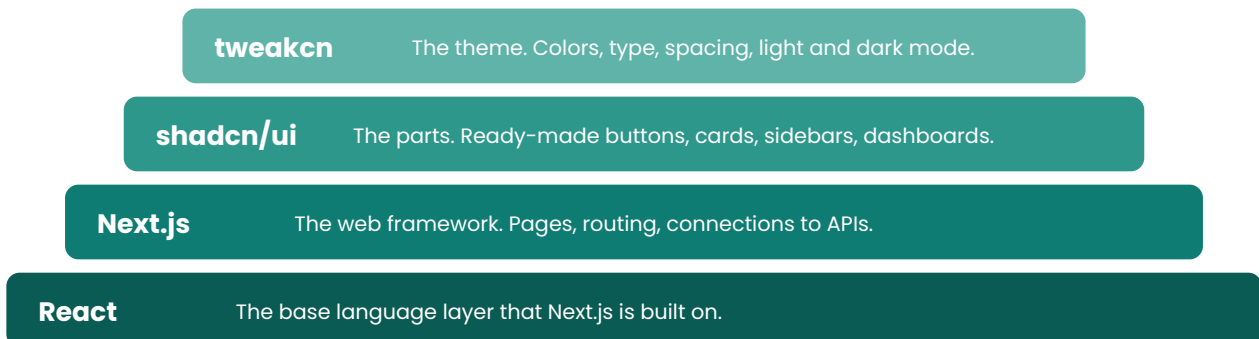
tweakcn is a design layer on top of shadcn/ui. It shows the same dashboard and lets you switch the whole look with one click, in light or dark mode. He browsed several themes, including one named Claude, a claymorphism theme he likes, and a pink one named Bubblegum. He chose a haze theme for the build. You can also make your own theme.

### THEMES ARE DESIGN SYSTEMS, NOT COLORS

A theme sets color, type, spacing and shape across every part of the app. That is why a themed app looks designed and not like a typical vibe-coded app.

## How the Layers Stack

He tested the group by asking for the order in the chat. Several learners got it right.



## How to Bring Them Into a Project

- On tweakcn, pick a theme. Click the Code button. Copy the install command.
- On shadcn/ui, open the block you want. Copy its install command.
- Paste both commands into the Lovable chat and ask it to install them.
- The command pulls in the layout, the theme and the code behind them.
- No plug-in is needed. It is all done through the prompt.
- He shared both links in the chat and told learners to explore the sites.

He made a wider point here. Anyone can vibe code now. Many vibe-coded apps look vibe-coded. A client product must look finished, and every button must work. He showed a real client dashboard as proof. It had a knowledge graph of the client's documents, a chatbot the client named Yoda, and connections to QuickBooks, NetSuite and Microsoft tools.

---

## The Product Idea

### A Personal Productivity CRM Dashboard

He explained the idea with his own case. His newsletter data and other work data sit in many places. He wants one dashboard where he can see everything. He described it as his own operating system, with voice control.

### Why This Product

- It is his most repeated client project, so the lessons are real.
- It is modular. The same code can be reused for other clients.
  - He has sold it to a signage and LED lighting business.
  - He has sold it as a personal AI agent.
  - He has sold it as a real estate dashboard.
- A learner noted that a CRM dashboard was already built in the earlier bootcamp.
  - He said that one was a basic introduction.
  - This one has voice, an action agent and real integrations. He called it production grade.

---

## Build Stage: Foundation in Plan Mode

### Start in Plan Mode

His first rule in Lovable: switch to plan mode before you build anything. In plan mode the tool asks questions and writes a plan. It does not write code yet. He spoke his prompt aloud and let it transcribe.

#### FIRST PROMPT, CLEANED UP

*I want to build a complete dashboard project. Use Next.js for the front end. Use Node.js for the back end. Use Supabase for the database. Also install the shadcn/ui library.*

### Lovable Pushed Back on the Stack

- Lovable replied that Next.js is not supported on its platform.
- Its projects run on React with server-side code and Lovable's own cloud back end.
- It said shadcn/ui was already available.
- He asked for Next.js once more. Lovable refused again and named React and TanStack.
- He accepted the supported stack. His reason: Next.js is built on React, so the result is the same kind of app.
- His advice still stands for other tools. On Bolt, ChatGPT or Claude, ask for this stack by name.

A stack, he explained, is just the full set of building blocks a project uses.

## Planning Questions and the Group's Choices

| Lovable asked                                | Choice made                                                                                                                        |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| How should users sign in?                    | Google sign-in. The chat voted for it.                                                                                             |
| Any visual preference?                       | Light and airy. The chat voted for it.                                                                                             |
| What is the dashboard for?                   | A professional personal CRM and operating system with voice commands. He told it to ignore features for now and focus on the plan. |
| Should each signed-in person have a profile? | Yes, with profiles. The chat voted for it.                                                                                         |

## The First Plan

Lovable returned a written plan. He read it aloud with the group before approving it.

- Sign-in.
- A clear dashboard home to return to. He called this important.
- A contacts list.
- An interactions list.
- Settings.
- A light theme.
- Technical notes and a build order.

The plan also had a section for things not in this phase: voice commands, pipelines, deals, reminders and email integrations. It said the data model leaves room for them.

### NEGATIVE PROMPT

A negative prompt tells the AI what not to do. The not-in-this-phase list works as one. It keeps the AI on the current goal.

He approved the plan. He noted that a plan can be edited before approval, for example to add notes.

## Why Not Ask for Everything at Once

A learner asked this. His answer: if you load every feature into one request, the AI drifts from its real goal. Work must go in clear phases. Another learner added that doing everything at once defeats the purpose of an MVP. He agreed.

## Watching Lovable Work

After approval, Lovable switched to build mode on its own. While it worked, he opened its thinking and activity log and read it aloud.

- It set up social sign-in.
- It created a storage bucket.

- It imported Supabase from its integrations.
- It started on the contacts page.
- At one point it thought for 10 seconds and then wrote a prompt for itself.

His point: a non-coder can still follow what is going on by reading this log in plain terms. He reads it on purpose, because the AI can drift or make things up. It did so later in the build.

## First Preview

- Lovable named the app Orbit. The tagline was about keeping every relationship in orbit.
- The landing page looked poor and had some errors. He had not prompted for it, so he moved on.
- Get Started opened Google sign-in. He signed in and granted permissions.
- The dashboard loaded with a home page and an empty contacts page.
- It was basic. But the foundation was in place.

# Build Stage: Features in Phases

## Feature Brainstorm With the Group

While Lovable built the foundation, he went back to the whiteboard and collected features from the chat. He added 2 of his own first: the shadow sidebar layout, and a voice AI with full context, like Jarvis.

| Group         | Ideas raised in the chat                                                                                                                                                |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Work          | Tasks, projects, Trello or other project management links, reminders, calendar, activity tracking, automations, document upload, issue resolution, reports.             |
| Goals         | Goals for the year, month and week. Fun plans and a gratitude log.                                                                                                      |
| Money         | Payments, quotes and invoices, expense tracking, daily expenses, payment reminders, a monthly investment plan, streaming and other subscriptions, stock market updates. |
| Clients       | Client names, projects, opportunities, lead generation, pipeline progress, policies.                                                                                    |
| Communication | Writing emails with AI, a personal email manager, WhatsApp messages, blogs, social media.                                                                               |
| Information   | Daily news updates, such as AI news in one place.                                                                                                                       |

## What He Set Aside

- Health and fitness tracking. A good idea and one he is building himself, but it belongs in a separate app.
- Full subscription management. He said it is a whole dashboard by itself.
- Social media. He asked the group to think about what fits a productivity tool.

He admitted his picks could be his own bias. There is no right or wrong feature list. If learners dislike his dashboard, they should drop it and keep the method.

## From Whiteboard Screenshot to a 3-Phase Plan

He explained that features must be grouped. Project management and quotes do not belong in the same step as stock market updates. Then he took a screenshot of the whiteboard, pasted it into Lovable, switched back to plan mode, and spoke the prompt. No typing or copying of the list was needed.

### PHASING PROMPT, CLEANED UP

*Look at the screenshot I attached. It has many features and requirements. Create a complete 3-phase plan. I want to see what to build now, what to build next, and what comes after. Sort every feature into a phase. Do not miss any of them. They are all important.*

He held back 2 things on purpose: the voice agent and the shadcn layout. He wanted the plan first.

| Phase   | Theme                         | Contents                                                                          |
|---------|-------------------------------|-----------------------------------------------------------------------------------|
| Phase 1 | Core workspace                | Tasks, projects, notes, calendar, goals, reminders, and a rebuilt home dashboard. |
| Phase 2 | Money and AI                  | Expense tracker, payment management and the other money and AI items.             |
| Phase 3 | Outside world and connections | Integrations and outside sources. The details were not read out.                  |

He approved it. Lovable built Phase 1 only. A learner asked why there was no WhatsApp button. His answer: that sits in a later phase, and only Phase 1 was live. Phases 2 and 3 were not built in the session because of time. They remain in the project chat, ready to run.

## Phase 1 Result

The preview now showed tasks, projects, calendar, notes, goals, reminders, contacts and settings. The group liked it. One learner said it could be smoother. He agreed and said the design prompt would fix that. He noted that this used to take engineers a long time.

## Build Stage: Applying the Design System

### DESIGN PROMPT, CLEANED UP

*This is great, but I want to improve my dashboard. Here is the exact shadcn dashboard I want to use. Install it using this command. I am also attaching the design system. Install the design system properly as well.*

- He pasted the 2 install commands with the prompt.
- He did not use plan mode this time. The job was simple. Planning it would waste credits.

- He expected a possible error, because the block is written for Next.js. Lovable handled it.
- He sent the prompt just before a 10-minute break, so Lovable worked while everyone rested.

After the break, Lovable reported that the dashboard now used the shadcn dashboard layout with a collapsible sidebar. The theme was applied across the whole app. He pointed out that the break time was not wasted. Talking with the group, collecting ideas and letting the tool build in the background is what he means by vibing. Some pages were still empty. He said they were one prompt away from being filled.

## Build Stage: The Jarvis Voice Agent

### Why Voice

He believes the way people use software is changing. Clicking a button and typing a name, email, phone and company into a form is the old way. The new way is to say what you want and let the app fill it in. He thinks voice comes first and gesture control comes after. He tried to demo hand tracking in his own workspace. It did not load, because he was on a different computer. He said gesture control needs only a camera and a server. His runs on a VPS.

### The Jarvis Prompt

#### VOICE AGENT PROMPT, CLEANED UP

*The dashboard is good. I want a few changes. Rename it from Orbit to Jarvis.*

*Add a right sidebar with a simple orb. Keep the main dashboard clear. Power the orb with the ChatGPT realtime API.*

*This must not be simple voice chat. It must be an action model. When I ask it to create a task, it adds the task. When I ask for a new project or goal, it creates it. Give me a complete plan for this.*

He admitted the prompt was long and loose. His point: even when your thoughts are racing, you can speak them out and the tool will sort them.

| Lovable asked                               | Choice made                                                 |
|---------------------------------------------|-------------------------------------------------------------|
| How do you want to talk to Jarvis?          | Voice and typing. He later called this a mistake.           |
| Should Jarvis speak its answers?            | Yes. Speak and show text.                                   |
| Should Jarvis ask before it changes things? | Confirm deletions only. Asking every time would be tedious. |

### The Plan Lovable Wrote

- Rename Orbit to Jarvis.
- Add a right sidebar with the orb.
- Let Jarvis create tasks, projects and notes.
- Send the voice recording to the back end.

- Use an OpenAI model through Lovable's AI gateway for reasoning.
- Use Gemini for speech to text.

While it built, the log showed a Supabase client being set up for each request with the user's JWT. Nobody on the call was sure of the term at first. The chat confirmed it means JSON Web Token, a token that proves who the user is. The log also showed the voice path: record audio in the browser, convert it to a WAV file, upload it for transcription, then speak the reply. He flagged this at once as a long chain that would feel slow.

## First Test: Review and Reject

The orb appeared in the right panel. But speaking to it only produced text in a chat box. It was a transcription tool, not a two-way voice agent. He said he did not like it at all, and went back to the loop diagram. This was the review step.

### KNOW WHICH API FITS THE JOB

Speech to text plus text to speech is a slow chain. A speech-to-speech model hears and answers directly. For a live voice agent, that is the right type.

He found the right one by searching for OpenAI's voice API and reading the OpenAI blog. He said following that blog and Twitter is how he stays current.

## Second Prompt: Refine

### REFINE PROMPT, CLEANED UP

*The orb section is nice. But I do not want to see text. Remove the chat box, the delete icon, the enter icon, the speaker icon and the mic icon. Keep only the orb. Also test the GPT realtime API, because it is a better speech-to-speech option. I attached a Jarvis image. Use it for the Jarvis interface.*

- The image idea came from a learner who asked for a Jarvis icon.
- Lovable checked its gateway and said no realtime audio model was available there.
- It said plainly that true speech to speech was not possible inside the gateway.
- It kept the faster loop of record, transcribe, act, speak.
- It rebuilt the panel as a HUD, a heads-up display, like the one inside the Iron Man helmet.
- The result looked much better. It still needed a tap to speak.
- A learner rated it 7 out of 10. He agreed.

## The Working Test

He asked Jarvis to add a task: write a newsletter and send it on Monday, September 21. Jarvis replied that it had added the task, due Monday, September 21 at 9:00 AM. The task did not show at first. After a reload it was there in the task list. He asked the chat for a boom. 2 years ago he could only have dreamed of this.

## What This Opens Up

- The automations and agents built elsewhere in the program can be connected to this dashboard.
- They can then be started by voice.
- True speech to speech needs your own OpenAI key. That is covered under API keys.
- Small voice models that run on your own computer exist. He named Qwen TTS. They are slower for now, but improving.

During this stage a learner shared a line credited to Naval Ravikant: vibe coding is more addictive than any video game, if you have something you want to build. Harshith agreed. He added that he had just bought a 3D printer and planned to try vibe coding physical designs.

## Build Stage: Integrations

A dashboard with no connections has little value. So the next stage was integrations. He noted that the group had listed features but had not listed integrations.

### INTEGRATION PROMPT, CLEANED UP

*I want to work on integrations now. I can see the calendar, but it is blank. Connect my Google Calendar. My whole Google Calendar should show here. Also add my Gmail. I have Slack too, but I will connect that later. For now, only Google Calendar and Gmail.*

## Google Calendar and Gmail

- Lovable asked if this was for his own Google account only. He said yes, for now.
- It asked if Gmail should be read only, or read and send. He chose read and send.
- He clicked connect, picked his personal Google account and granted access.
  - Access covered viewing email and settings, managing drafts and sending.
- Lovable noted a limit on shared workspaces. Only workspace members can use the connection.
- The log showed it merging Google Calendar events with local events into one view.
- It set new events to be written to Google Calendar by default.
- It added a mail page and a link to it in the sidebar.

## Handling Errors

An error screen appeared during this stage. His fix was simple. Click the button that offers to fix the error and let Lovable repair it. He also mentioned earlier that you can paste documentation into the chat when the tool is stuck on a connection.

## Result

The calendar showed his personal events. It was his personal account, so no work events appeared. The mail page showed his Gmail inbox inside the dashboard. He no longer needs to open Gmail. He can ask Jarvis for the latest email and tell it to reply. He joked that work has become putting on earphones and saying reply to him, invoice them.



## WhatsApp

- Many learners asked about WhatsApp, including reading group updates.
- He said it is possible but not as direct as Google.
- It uses an open-source WhatsApp MCP server. He shared the link in the chat.
- Steps: paste the link into Lovable, ask it to integrate your WhatsApp, then scan the QR code it shows.
- A learner asked if this could get an account banned. He said no.
  - His reason: it is only a bridge. Messages are stored locally on your laptop and streamed to the app.

## Other Integrations Mentioned

- **Stripe.** Easy to connect. The documentation is simple. You could ask Jarvis about recent payments or invoices.
- **Slack.** Planned for later.
- **Your own automations.** Workflows and agents from the program can be plugged in.

### FIND YOUR TOOLS

His view: the hard part is no longer how to integrate. It is finding the set of tools that work for you. Once you find one, copy its link, paste it into Lovable and ask for the integration.

## A Side Demo of His Own Jarvis

To show the range of what is possible, he asked his own desktop Jarvis to check his work inbox. It answered that no reviewed email adapter was set up, and that only browser control was ready. He told it to open a new chat and plan the integration. He then compared the 2 builds. His took about 4 weeks. The session build took about 3 hours. He said learners who spend 3 weeks on this could do better than him, because many have deeper business experience.

# API Keys, Credits and Cost

## How Credits Work in Lovable

- Every prompt uses credits. Planning uses credits too.
- Voice features use credits as well.
- Skip plan mode for small, simple changes.
- Lovable shows the AI requests, the credits used and the request rate for the project.

## Why No API Key Was Needed

He opened the project code and the environment variables. No OpenAI key had been added at any point. The voice and reasoning features ran on Lovable's own AI gateway. Lovable also handled the Gemini connection in the back end.

**NEVER REVEAL ENVIRONMENT VARIABLES**

The ENV file holds the secret keys for services such as Supabase. He said these must never be shown or shared.

## Getting Your Own Key

- Go to [platform.openai.com](https://platform.openai.com).
- Add a card and load some credit. He suggested \$10 to start.
  - That is enough to test fully and to prove the MVP to a client.
- Open API Keys and create a new key. Give it a name. In the demo he set it to expire after 1 day.
- The same steps apply on the Claude platform. OpenRouter is another source.
- While showing his own account he was surprised to see he had spent about \$1,000 on API credits.

**PROMPT TO ADD YOUR OWN KEY**

*Add my OpenAI API key. I will give my key. Make the voice version work with the speech-to-speech realtime API.*

Lovable then shows a small secure field for the key. You paste it in and the feature is wired up. He named OpenAI's realtime models, and a newer release he called GPT Live, as the better options for speech to speech.

## BYOK: Bring Your Own Key

When you sell to a client, write BYOK into the contract. The client brings and pays for their own API key. You deliver the best dashboard you can. You do not carry their usage costs.

## Total Cost of the Build

Learners asked how many credits and how much money the build used. He could not say during the session, because the build was still running. He promised the numbers in the resource document.

# Publishing

- He clicked Publish in Lovable and opened the live URL.
- The app was up and running.
- The landing page was still weak. The session focused on the dashboard, and the landing page can be improved with more prompts.
- He will put the full source code on GitHub and share it.
- Learners can build on top of it and use it every day.
- With the source code, you do not have to publish on Lovable. You can host on Vercel or elsewhere.
- In that case you only pay for your own API usage, for the voice model and the dashboard.

---

## Turning This Into Income

He was direct about this. He wants learners to make money from what they build. Many businesses want AI coworkers, agents and personal dashboards.

### Pricing

- He said this codebase, offered through a consulting website, can sell for a minimum of \$2,000.
- His own standard offer is an 8-week implementation plan for \$6,000.
- After the 8 weeks, further work is on a retainer.
- What you charge depends on your expertise and how you package the offer.

### 3 Ways to Package It

| Model                     | How it works                                                          |
|---------------------------|-----------------------------------------------------------------------|
| Charge for implementation | The dashboard stays yours. The client pays you to set it up for them. |
| Charge for maintenance    | You provide the dashboard and the setup. The client pays for upkeep.  |
| Sell it outright          | The client buys the whole dashboard.                                  |

### Who Owns the Database and the IP

A learner asked this. His answer: it depends on how you package the service and what the client wants. In one real project he built a coworker dashboard and an agent. He installed the memory and knowledge base on the client's server. He kept ownership of the dashboard. IP means intellectual property.

### Service as a Software

He asked what SaaS stands for. The chat answered software as a service. Before AI, that meant building one product, such as Salesforce, Zoho or QuickBooks, and charging everyone monthly. Now that dashboards are easy to build, he flips the words to service as a software.

- You build the dashboard and the agent once.
- You package it as a service and offer it to clients.
- You adjust it for each client. The same Jarvis, slightly different each time.
  - A real estate firm.
  - An IT company.
  - An accountant.
  - A digital billboard business.
  - A pharmaceutical client.
- The value is in the implementation and in the harness.
- By harness he means the layer under the product: the design, the setup and the connections.

He also shared a line he had read: AI vibe coding is a power tool for creative people who always wanted to build something and could not.

---

## Questions From the Chat

**Are you a techie?**

No. He said he only sounds like one because of the words he uses.

**Why not build the final product right now?**

No big product was built in one step. You ship a basic version and improve it in rounds.

**Why front end first and not back end?**

It is not one before the other. Both are built in parallel.

**If we build in Lovable, how do we connect Next.js, Node.js and Supabase?**

Through prompts. You name them and the tool sets them up.

**Is shadcn the same as shadcn/ui?**

Yes. The whole thing is one component library. It can be used for any component.

**Is anything plugged in to use shadcn/ui?**

No. It is all done through prompts.

**Where do we publish the site?**

Lovable can publish it.

**Does planning use Lovable credits?**

Yes. Every prompt does.

**Why not include every feature in the first plan?**

The AI loses focus when it gets too much at once. Work in phases.

**Why 3 phases?**

So each phase holds features that belong together. Tasks and projects do not belong with stock market updates.

**Is Orbit inside Lovable?**

Yes. Lovable chose the name. The group suggested Jarvis, Ultron and Vision. He went with Jarvis.

**Can we make custom themes?**

Yes. That is what tweakcn is for.

**Will the voice feature use credits?**

Yes.

**Do we need to connect our own key for Gemini or OpenAI?**

Not inside Lovable. It handles those connections. You can add your own key if you want to.

**Does gesture control need a special system?**

No. A camera and a server are enough.

**Can we use this CRM daily once it is built?**

Yes. He will share the source code so learners can use it and extend it.

Is the Google connection only for your own account?

Yes, for now.

Can it connect to WhatsApp, including groups?

Yes, through the WhatsApp MCP server and a QR code scan.

If I sell the dashboard, how much do I charge?

It depends on your expertise. His standard is \$6,000 for 8 weeks, then a retainer.

If we sell this, who owns and runs the database?

It depends on the package you agree with the client.

---

## Common Mistakes and Misconceptions

- **Building along while trying to learn the method.** You miss steps and burn credits.
- **Thinking you must know how to code.** You need to know what the blocks are, not how to write them.
- **Thinking the method belongs to one tool.** It works on any vibe coding tool.
- **Skipping plan mode on big work.** The plan is where you catch problems cheaply.
- **Using plan mode on small work.** It wastes credits.
- **Asking for everything in one prompt.** The AI drifts. Use phases.
- **Mixing unrelated features in one phase.** Group what belongs together.
- **Describing components from scratch.** Ask for shadcn/ui parts by name.
- **Treating a theme as a color choice.** It is a full design system.
- **Using Google Sheets as the app database.** Use a real database such as Supabase.
- **Not reading what the AI is doing.** Open its thinking. It can drift or make things up.
- **Picking the wrong type of API.** A live voice agent needs speech to speech, not 2 separate steps.
- **Accepting the first output.** Review it. Reject it if it is wrong. Refine.
- **Showing environment variables.** They hold secret keys.
- **Shipping something that looks vibe-coded.** Client work must look finished and every button must work.
- **Paying for the client's AI usage.** Use BYOK.

---

## Action Steps

- Watch the recording once without building. Note how each prompt is worded.
- Pick one idea from your own list. Write down what it is and who it is for.
- Sketch the parts on paper: front end, back end, database, APIs.
- Explore the shadcn/ui blocks and the tweakcn themes. Pick one layout and one theme.
- Open your tool in plan mode. Name the stack you want. Let it ask questions.
- Read the plan. Check the not-in-this-phase list. Edit it if needed. Then approve.
- List every feature you want. Take a screenshot. Ask for a 3-phase plan. Build Phase 1 only.

- Paste the install commands for the layout and the theme. Skip plan mode for this.
- Add a voice action agent. Start with creating tasks, projects and notes. Confirm deletions only.
- Connect one integration first, such as Google Calendar or Gmail.
- When you see an error, use the fix button. Paste documentation if the tool is stuck.
- Publish. Test every page and every button.
- Get your own API key when you need speech to speech or when you move off Lovable.
- Decide how you would package it for a client: implementation, maintenance or outright sale.
- Collect the prompts, slide deck, links and source code from the LMS when they are posted.
- Save credits for the hackathon.

---

## Key Takeaways

- Vibe coding is prompting. You describe the product and the AI builds it.
- An MVP is the most basic working version you can ship. Improve it in rounds.
- Every build is a loop: prompt, generate, review, refine.
- Learn the blocks: Next.js, Node.js, Supabase, shadcn/ui, tweakcn. They are open source.
- Plan first. Build in phases. Tell the AI what is out of scope.
- Do not rebuild what already exists. Install layouts and themes with one command.
- Voice is becoming the way people use software. An action agent turns a dashboard into a product.
- Know which API fits which job. Follow the AI labs' blogs to stay current.
- Integrations are now easy. Finding the right tools is the real work.
- The same codebase can be adjusted and sold to many kinds of clients. He calls this service as a software.

---

## Closing and What Comes Next

### Session Close

Harshith thanked the group, the co-mentors and the team. He hoped the session had opened new ideas for learners to explore with their own projects. He shared his social links in the chat. Sumedha opened the feedback poll and asked learners to post their thanks in the channel and the chat.

### Admin Notes

- The group voted for no break before the breakout rooms.
- The reflection link and webinar ID were shared in the chat. The reflection link is the same as the badge link.
- Batches are created per day, so the voice agent session and this session share one batch.
- Breakout Q and A ran for about 1 hour. The day closed around 8:00 PM.
- Learners with no questions were free to leave.

- Recordings and resources go into the LMS the same night. Vibe coding resources sit below the voice agent resources.
- A feedback form for the Q and A was shared at the end.

## Plan for the Next Day

- A full-day session starting at 10:00 AM IST.
- First session: Connecting the Dots with Vaibhav Sisinty. He will tie together everything learned so far and give learners a plan.
- Then a lunch break.
- Then Building Jerry. A mentor named Akhil teaches the learning session.
- Co-mentors then help learners build their own Jerry step by step in breakout rooms.
- Jerry is the in-house name for a multi-agent AI system. The name comes from Vaibhav's own agent. Learners can name theirs anything.
- By the end of that day, each learner should have their own Jerry and a clear plan.

## Glossary

| Term             | Plain meaning                                                        |
|------------------|----------------------------------------------------------------------|
| Vibe coding      | Building software by telling an AI what you want.                    |
| MVP              | Minimum viable product. The most basic working version you can ship. |
| Ship             | To deliver a product to real users.                                  |
| Proof of concept | A small build that shows an idea works.                              |
| Wireframing      | Mapping out screens and parts before building.                       |
| Stack            | The full set of building blocks a project uses.                      |
| Front end        | The part of an app the user sees.                                    |
| Back end         | The logic that runs behind the screen.                               |
| Database         | Where the app stores what it must remember.                          |
| API              | A connector that lets 2 systems talk to each other.                  |
| API key          | A secret code that lets your app use a paid service.                 |
| BYOK             | Bring your own key. The client supplies and pays for their API key.  |
| ENV variables    | A private file of settings and secret keys.                          |
| JWT              | JSON Web Token. A token that proves who a signed-in user is.         |

| Term             | Plain meaning                                                         |
|------------------|-----------------------------------------------------------------------|
| Component        | One piece of an interface, such as a button or sidebar.               |
| Block            | A full ready-made layout in shadcn/ui.                                |
| Design system    | A set of rules for color, type, spacing and shape.                    |
| Plan mode        | A Lovable mode that asks questions and writes a plan before any code. |
| Negative prompt  | An instruction about what not to do.                                  |
| Action model     | An assistant that does things in the app, not one that only talks.    |
| Speech to speech | A voice model that hears and answers directly, with no text step.     |
| HUD              | Heads-up display. The style used for the Jarvis panel.                |
| Orb              | The round visual that stands for the voice assistant.                 |
| MCP server       | A bridge that lets an AI tool use another service, such as WhatsApp.  |
| Harness          | The layer under a product: design, setup and connections.             |
| IP               | Intellectual property. Who owns what was built.                       |
| CMS              | Content management system.                                            |
| VPS              | A rented private server.                                              |